OBJECT DETECTION USING YOLO ALGORITHM

Professional Project Report

Author: Abdullah Shaheer

Technical Scope Note: The current implementation is a simplified YOLO-like CNN for educational purposes, not an official YOLO release.

Abstract

This project presents an educational implementation of an object-detection pipeline inspired by the YOLO approach. The system uses TensorFlow/Keras to build a convolutional neural network that predicts a normalized bounding box and a class value from synthetic images. OpenCV is used to generate the rectangular objects, NumPy handles numerical data, and Matplotlib is used for visualization.

1. Introduction

Object detection is a computer-vision task that identifies objects and determines their locations in an image. Unlike image classification, object detection also predicts spatial information. This project demonstrates these concepts through a simplified YOLO-like learning pipeline.

2. Problem Statement

The objective is to generate synthetic object-detection data, represent objects using normalized bounding boxes, train a CNN to predict the bounding-box values, evaluate the model, and measure overlap using IoU.

3. Objectives

• Understand object-detection data representation.

• Practice bounding-box normalization.

• Build a CNN using TensorFlow/Keras.

• Train a regression-style detector.

• Implement IoU.

• Compare predictions with ground truth.

4. Technologies

Python, TensorFlow/Keras, NumPy, OpenCV, and Matplotlib.

5. Dataset

The program creates 200 samples by default. Each sample is a 128×128 RGB image containing one randomly positioned filled white rectangle on a black background. The label is [x_center, y_center, width, height, class].

6. Model Architecture

The CNN contains Conv2D layers with 16, 32, and 64 filters, MaxPooling layers, Flatten, a 128-neuron Dense layer, and a final Dense layer with five outputs.

7. Training

The model uses the Adam optimizer and mean squared error loss. Training is configured for 10 epochs with a batch size of 32 and a 20% validation split from the training set.

8. Evaluation

The script evaluates held-out test data and predicts five test examples. Each prediction is compared with its corresponding ground-truth bounding box using IoU.

9. IoU

Intersection over Union is calculated as Intersection Area divided by Union Area. It measures the overlap between a predicted bounding box and a ground-truth bounding box.

10. Results

The actual numerical loss and IoU values depend on executing the program because the dataset is randomly generated. The documentation does not invent performance values.

11. Limitations

The current implementation uses synthetic data, one object per image, one class, a simplified CNN, and a single bounding-box output. It does not implement official YOLO grid prediction, objectness loss, Non-Maximum Suppression, multi-scale detection, or pretrained YOLO weights.

12. Future Work

Future extensions can include real annotated datasets, multiple objects and classes, confidence prediction, NMS, data augmentation, detection-specific losses, mAP/precision/recall evaluation, and webcam/video inference.

13. Conclusion

The project provides a compact educational demonstration of image preparation, bounding-box labels, CNN feature extraction, regression-style prediction, model evaluation, and IoU measurement.